ユーザー編集ダイアログUI設計書

メタデータ

title: ユーザー編集ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-02-24
author: Development Team
tags: [system-admin, user-management, dialog, edit-form]

関連ドキュメント

- ユーザー管理画面
- ユーザー追加ダイアログ
- ユーザー管理API仕様
- UIポリシー

---

1. 概要

1.1. 目的

システム管理者が既存ユーザーの情報を編集するためのモーダルダイアログです。ユーザー名、所属部署、ロール、二段階認証設定、アクティブ状態を変更できます。

1.2. 表示条件

- ユーザー管理画面の「編集」ボタンをクリック
- System Admin権限が必要

1.3. 画面イメージ



---

2. ダイアログ構成

2.1. レイアウト

```
┌──────────────────────────────────────┐
│ ユーザー編集                    [×]  │
├──────────────────────────────────────┤
│                                      │
│  ユーザー名 *                        │
│  ┌──────────────────────────────┐    │
│  │ 山田 太郎                    │    │
│  └──────────────────────────────┘    │
│                                      │
```

```
│  メールアドレス                                  │
│  ┌──────────────────────────────────────┐  │
│  │ user001@example.com（読み取り専用）  │  │
│  └──────────────────────────────────────┘  │
│  ※ユーザーIDのため変更できません                │
│                                                 │
│  所属部署 *                                     │
│  ┌──────────────────────────────────────┐  │
│  │ 開発部                          ▼    │  │
│  └──────────────────────────────────────┘  │
│                                                 │
│  ロール *                                       │
│  ┌──────────────────────────────────────┐  │
│  │ システム管理者                  ▼    │  │
│  └──────────────────────────────────────┘  │
│                                                 │
│  二段階認証設定 *                               │
│  ┌──────────────────────────────────────┐  │
│  │ 済                              ▼    │  │
│  └──────────────────────────────────────┘  │
│                                                 │
│  アクティブ判定 *                               │
│  ┌──────────────────────────────────────┐  │
│  │ アクティブ                      ▼    │  │
│  └──────────────────────────────────────┘  │
│                                                 │
├─────────────────────────────────────────────┤
│                    [キャンセル]  [保存]  │
└─────────────────────────────────────────────┘
```

2.2. モーダル仕様

コンポーネント: `<Modal>`

仕様:

項目	値
タイトル	「ユーザー編集」
幅	600px
位置	画面中央
マスク	背景を暗くする
キーボードESC	閉じる（確認なし）
マスククリック	閉じない

3. 入力項目

3.1. ユーザー名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「ユーザー名」
必須	○
プレースホルダー	「山田 太郎」
最大文字数	30文字
初期値	選択したユーザーのユーザー名

バリデーション:

```
{
  required: true,
  message: 'ユーザー名を入力してください'
}
```

3.2. メールアドレス（読み取り専用）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「メールアドレス」
必須	-
読み取り専用	○
初期値	選択したユーザーのメールアドレス

背景色	グレー（読み取り専用を示す）

注意事項:

- **編集時はメールアドレス変更不可**（主キーのため）
- `disabled={true}` を設定
- ユーザーIDとして使用されているため、変更すると整合性が崩れる

実装例:

```
<Form.Item
  name="mailAddress"
  label="メールアドレス"
>
  <Input
    placeholder="example@company.com"
    disabled={true}
  />
</Form.Item>
```

3.3. 所属部署（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「所属部署」
必須	○
プレースホルダー	「所属部署を選択」
データソース	departmentApiService.fetchMasters()
初期値	選択したユーザーのdepartment_cd

選択肢:

```
[
  { value: 'DEPT001', label: '開発部' },
  { value: 'DEPT002', label: '営業部' },
```

```
  // ...
]
```

バリデーション:

```
{
  required: true,
  message: '所属部署を選択してください'
}
```

3.4. ロール（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「ロール」
必須	○
プレースホルダー	「ロールを選択」
データソース	roleApiService.fetchMasters()
初期値	選択したユーザーのrole_cd

選択肢:

```
[
  { value: 'ROLE001', label: 'プロバイダー' },
  { value: 'ROLE002', label: 'システム管理者' },
  { value: 'ROLE003', label: 'データ管理者' },
  { value: 'ROLE004', label: '実験管理者' },
  { value: 'ROLE005', label: '実験作業者' },
  { value: 'ROLE006', label: 'MI技術者' },
]
```

バリデーション:

```
{
  required: true,
  message: 'ロールを選択してください'
}
```

3.5. 二段階認証設定（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「二段階認証設定」
必須	○
初期値	選択したユーザーのtwo_factor_auth（0または1）

選択肢:

```
[
  { value: 0, label: '未' },
  { value: 1, label: '済' },
]
```

注意事項:

- user_authenticationテーブルのentry_flgに対応
- 0 = false（未設定）、1 = true（設定済み）
- 編集時は変更可能だが、user_authenticationテーブルは更新しない（organization_userテーブルのみ更新）

バリデーション:

```
{
  required: true,
```

```
  message: '二段階認証設定を選択してください'
}
```

3.6. アクティブ判定（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「アクティブ判定」
必須	○
初期値	選択したユーザーのis_active（trueまたはfalse）

選択肢:

```
[
  { value: true, label: 'アクティブ' },
  { value: false, label: '非アクティブ' },
]
```

バリデーション:

```
{
  required: true,
  message: 'アクティブ判定を選択してください'
}
```

---

4. アクションボタン

4.1. 配置

位置: モーダルフッター（右寄せ）

レイアウト:

```
<Space>
  <Button onClick={handleModalCancel}>キャンセル</Button>
  <Button type="primary" onClick={handleModalOk} loading={loading}>保存</Button>
</Space>
```

4.2. キャンセルボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「キャンセル」
タイプ	`default`

動作:

1. モーダルを閉じる
2. フォームをリセット
3. 確認なし

4.3. 保存ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「保存」
タイプ	`primary`
ローディング	送信中は `true`
無効化条件	loading=true

動作フロー:

1. フォームバリデーション実行
2. バリデーションエラーがある場合、エラーメッセージ表示
3. ローディング状態に変更
4. **organization_userテーブルをUPDATE**（パスワード関連は更新しない）
5. 成功時:
    - モーダルを閉じる
    - データ再読み込み
    - 成功メッセージ表示: `"ユーザー情報を更新しました"`
6. 失敗時:
    - エラーメッセージ表示
    - モーダルは開いたまま

実装例:

```
const handleModalOk = async () => {
  try {
    const values = await form.validateFields();

    // 更新データ（パスワードは含まない）
    const updateData: UpdateUserRequest = {
      userName: values.userName,
      departmentCd: values.departmentCd,
      roleCd: values.roleCd,
      isActive: values.isActive,
    };

    setLoading(true);
    await updateUser(String(editingUser.userId), updateData);
    await loadUsers();
    message.success('ユーザー情報を更新しました');

    setIsModalVisible(false);
    form.resetFields();
  } catch (error) {
    console.error('保存に失敗:', error);
    const errorMessage = error instanceof Error ? error.message : '保存に失敗しました';
    message.error(errorMessage);
  } finally {
    setLoading(false);
```

```
  }
};
```

---

5. 初期値の設定

5.1. ダイアログ表示時の処理

トリガー: 編集ボタンクリック

処理フロー:

1. 編集対象のユーザーをstateに保存
2. フォームに初期値を設定
3. モーダルを表示

実装例:

```
const handleEdit = (user: UserRecord) => {
  setEditingUser(user);
  form.setFieldsValue({
    userName: user.userName,
    mailAddress: user.mailAddress,          // 読み取り専用
    departmentCd: user.departmentCd,
    roleCd: user.roleCd,
    twoFactorAuth: user.twoFactorAuth ? 1 : 0,  // boolean → number
    isActive: user.isActive,
  });
  setIsModalVisible(true);
};
```

---

6. API仕様

6.1. ユーザー更新

エンドポイント: `PUT /api/v1/systemManagement/user/:userId`

パスパラメータ:

パラメータ	型	必須	説明
userId	string	○	ユーザーID（メールアドレス）

リクエストヘッダー:

```
Content-Type: application/json
Authorization: Bearer {session_token}
```

リクエストボディ:

```
{
  "userName": "山田太郎（更新）",
  "departmentCd": "DEPT002",
  "roleCd": "ROLE002",
  "isActive": true
}
```

注意事項:

- パスワードフィールドは含まない
- メールアドレス（userId）は変更不可

レスポンス（成功時）:

```
{
  "success": true,
  "data": {
    "userId": "user001@example.com",
    "userName": "山田太郎（更新）",
    "mailAddress": "user001@example.com",
    "departmentCd": "DEPT002",
    "departmentName": "営業部",
    "roleCd": "ROLE002",
    "roleName": "システム管理者",
    "twoFactorAuth": true,
    "isActive": true,
    "updateAt": "2026-02-24T08:50:00Z"
  },
```

```
  "message": "ユーザー情報を更新しました"
}
```

レスポンス（エラー時 - ユーザーが存在しない）:

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "ユーザーが存在しません"
  },
  "meta": {
    "timestamp": "2026-02-24T08:50:00Z",
    "requestId": "req_abc123"
  }
}
```

---

7. バリデーション

7.1. クライアントサイドバリデーション

必須チェック:

- ユーザー名
- 所属部署
- ロール
- 二段階認証設定
- アクティブ判定

注意事項:

- メールアドレスは読み取り専用のため、バリデーション不要
- パスワード関連のバリデーション不要

7.2. サーバーサイドバリデーション

存在チェック:

- ユーザーID（メールアドレス）が存在するか

データ整合性チェック:

- 所属部署CDが有効か
- ロールCDが有効か

---

8. エラーハンドリング

8.1. エラーメッセージ一覧

エラーコード	メッセージ	表示場所
`VALIDATION_ERROR`	「ユーザー名を入力してください」	フォームフィールド
`VALIDATION_ERROR`	「所属部署を選択してください」	フォームフィールド
`VALIDATION_ERROR`	「ロールを選択してください」	フォームフィールド
`RESOURCE_NOT_FOUND`	「ユーザーが存在しません」	トースト
`SYSTEM_ERROR`	「保存に失敗しました」	トースト

8.2. エラー表示方法

フォームフィールドエラー:

```
form.setFields([
  {
    name: 'userName',
    errors: ['ユーザー名を入力してください'],
  },
]);
```

トーストメッセージ:

```
message.error('ユーザー情報の更新に失敗しました');
```

---

9. ユーザーフロー

9.1. 正常フロー

```
[ユーザー管理画面]
    ↓ 「編集」ボタンクリック（対象: user001@example.com）
[ユーザー編集ダイアログ] 表示
    フォームに初期値を設定:
      - ユーザー名: "山田 太郎"
      - メールアドレス: "user001@example.com"（読み取り専用）
      - 所属部署: "DEPT001"（開発部）
      - ロール: "ROLE002"（システム管理者）
      - 二段階認証設定: 1（済）
      - アクティブ判定: true（アクティブ）
    ↓ ユーザー名変更: "山田 太郎（更新）"
    ↓ 所属部署変更: "DEPT002"（営業部）
    ↓ 「保存」ボタンクリック
フォームバリデーション実行
    ↓ OK
API呼び出し: updateUser(userId, updateData)
    organization_userテーブルをUPDATE
    ↓ 成功
[ユーザー管理画面]
    データ再読み込み
    成功メッセージ表示: "ユーザー情報を更新しました"
```

9.2. エラーフロー（ユーザーが存在しない）

```
[ユーザー編集ダイアログ]
    ↓ フィールドを変更
    ↓ 「保存」ボタンクリック
フォームバリデーション実行
    ↓ OK
API呼び出し: updateUser(userId, updateData)
    ↓ エラー（404 Not Found）
[ユーザー編集ダイアログ]
```

```
エラーメッセージ表示："ユーザーが存在しません"
モーダルは開いたまま
```

---

10. データベース更新処理

10.1. 1テーブル更新

organization_userテーブル

```sql
UPDATE organization_user
SET
  user_name = $1,
  department_cd = $2,
  role_cd = $3,
  is_active = $4,
  update_user_id = $5,
  update_at = CURRENT_TIMESTAMP
WHERE
  organization_id = $6
  AND user_id = $7;
```

注意事項:

- user_authenticationテーブルは更新しない（パスワード変更機能は別途実装）
- メールアドレス（user_id）は変更不可

---

11. アクセシビリティ

11.1. キーボード操作

Tab順序:

1. ユーザー名入力
2. メールアドレス入力（読み取り専用、スキップ可能）
3. 所属部署選択

4. ロール選択
5. 二段階認証設定選択
6. アクティブ判定選択
7. キャンセルボタン
8. 保存ボタン

ショートカットキー:

- Enter: 保存ボタンのクリック（フォーカスが入力フィールドにない場合）
- Escape: モーダルを閉じる

11.2. ARIA属性

```
<Modal
  title="ユーザー編集"
  open={isModalVisible}
  onCancel={handleModalCancel}
  aria-labelledby="user-edit-dialog-title"
  aria-describedby="user-edit-dialog-description"
>
  <Form form={form} layout="vertical">
    <Form.Item
      label="ユーザー名"
      name="userName"
      rules={[{ required: true }]}
    >
      <Input
        aria-label="ユーザー名"
        aria-required="true"
      />
    </Form.Item>

    <Form.Item
      label="メールアドレス"
      name="mailAddress"
    >
      <Input
        disabled={true}
        aria-label="メールアドレス（読み取り専用）"
        aria-readonly="true"
      />
    </Form.Item>
```

```
  </Form>
</Modal>
```

---

12. セキュリティ考慮事項

12.1. パスワード管理

- **編集ダイアログではパスワード変更不可**
- パスワード変更機能は別途実装（パスワードリセット機能など）

12.2. マルチテナント分離

- organization_idは自動付与（authMiddleware）
- 他組織のユーザーは編集不可

12.3. 監査ログ

- update_user_idにログインユーザーIDを記録
- 更新日時を自動記録

---

13. 追加ダイアログとの違い

項目	追加ダイアログ	編集ダイアログ
タイトル	「ユーザー追加」	「ユーザー編集」
メールアドレス	入力可能（必須）	読み取り専用
パスワード入力欄	あり（必須）	**なし**
パスワード確認入力欄	あり（必須）	**なし**
初期値	空	選択したユーザーの情報

APIエンドポイント	`POST /api/v1/systemManagement/user`	`PUT /api/v1/systemManagement/user/:userId`
DB操作	2テーブルINSERT	**1テーブルUPDATE**
パスワードハッシュ化	実行	**実行しない**

---

14. パスワード変更について

14.1. 編集ダイアログでパスワード変更不可の理由

1. **セキュリティ**: パスワード変更は慎重に行うべき操作
2. **監査**: 別途パスワード変更履歴を記録する必要がある
3. **ユーザビリティ**: 一般的なユーザー情報編集とパスワード変更は別機能として実装

14.2. パスワード変更の実装方針

別途実装する機能:

- **パスワードリセット機能**: 管理者が対象ユーザーのパスワードをリセット
- **パスワード変更機能**: ユーザー自身が現在のパスワードを入力して新しいパスワードに変更

---

15. 実装例

15.1. 編集ダイアログのコンポーネント

```
// 編集ボタンクリック時の処理
const handleEdit = (user: UserRecord) => {
  setEditingUser(user);
  form.setFieldsValue({
    userName: user.userName,
    mailAddress: user.mailAddress,          // 読み取り専用
    departmentCd: user.departmentCd,
```

```
    roleCd: user.roleCd,
    twoFactorAuth: user.twoFactorAuth ? 1 : 0,  // boolean → number
    isActive: user.isActive,
  });
  setIsModalVisible(true);
};

// 保存ボタンクリック時の処理
const handleModalOk = async () => {
  try {
    const values = await form.validateFields();

    // 更新データ（パスワード不要）
    const updateData: UpdateUserRequest = {
      userName: values.userName,
      departmentCd: values.departmentCd,
      roleCd: values.roleCd,
      isActive: values.isActive,
    };

    setLoading(true);
    await updateUser(String(editingUser.userId), updateData);
    await loadUsers();
    message.success('ユーザー情報を更新しました');

    setIsModalVisible(false);
    form.resetFields();
  } catch (error) {
    console.error('保存に失敗:', error);
    const errorMessage = error instanceof Error ? error.message : '保存に失敗しました';
    message.error(errorMessage);
  } finally {
    setLoading(false);
  }
};

// フォーム（パスワード入力欄なし）
<Modal
  title="ユーザー編集"
  open={isModalVisible}
  onOk={handleModalOk}
  onCancel={handleModalCancel}
  width={600}
  okText="保存"
  cancelText="キャンセル"
  confirmLoading={loading}
>
  <Form form={form} layout="vertical">
    {/* ユーザー名（編集可能） */}
    <Form.Item
```

```
      name="userName"
      label="ユーザー名"
      rules={[{ required: true, message: 'ユーザー名を入力してください' }]}
    >
      <Input placeholder="山田 太郎" />
    </Form.Item>

    {/* メールアドレス（読み取り専用） */}
    <Form.Item
      name="mailAddress"
      label="メールアドレス"
    >
      <Input
        placeholder="example@company.com"
        disabled={true}
      />
    </Form.Item>

    {/* 所属部署 */}
    <Form.Item
      name="departmentCd"
      label="所属部署"
      rules={[{ required: true, message: '所属部署を選択してください' }]}
    >
      <Select
        placeholder="所属部署を選択"
        options={departments}
      />
    </Form.Item>

    {/* ロール */}
    <Form.Item
      name="roleCd"
      label="ロール"
      rules={[{ required: true, message: 'ロールを選択してください' }]}
    >
      <Select
        placeholder="ロールを選択"
        options={roles}
      />
    </Form.Item>

    {/* 二段階認証設定 */}
    <Form.Item
      name="twoFactorAuth"
      label="二段階認証設定"
      rules={[{ required: true, message: '二段階認証設定を選択してください' }]}
    >
      <Select
        options={[
```

```
            { value: 0, label: '未' },
            { value: 1, label: '済' },
          ]}
        />
      </Form.Item>

      {/* アクティブ判定 */}
      <Form.Item
        name="isActive"
        label="アクティブ判定"
        rules={[{ required: true, message: 'アクティブ判定を選択してください' }]}
      >
        <Select
          options={[
            { value: true, label: 'アクティブ' },
            { value: false, label: '非アクティブ' },
          ]}
        />
      </Form.Item>
    </Form>
  </Modal>
```

---

16. 変更履歴

| バージョン | 日付 | 変更内容 | 担当者